

PROJET DATA LAKE

Documentation Technique

Projet Data Lake Finance - installation, architecture et verification

Equipe	Artemie Smogunov, Nathan Smadja-Tubiana et Patrice Ignongui
Theme	Finance uniquement
Module	Data Lakes & Data Integration - EFREI 2025-2026
Professeur	Yvann Vincent
Date	7 juillet 2026

Validation technique effectuee sur Docker, Airflow, FastAPI, MinIO, Elasticsearch, PostgreSQL et Redis.

1. Architecture de la solution

La solution repose sur une architecture locale conteneurisee. Elle est pensee pour etre reproductible sur une machine disposant de Docker Desktop.

Service	Port	Role
FastAPI	8000	API Gateway et endpoints d ingestion.
Airflow	8080	Orchestration du pipeline financier.
MinIO	9000 / 9001	Stockage S3 compatible pour la zone Raw.
Elasticsearch	9200	Indexation et recherche dans la zone Raw.
PostgreSQL	5432	Tables Staging, Curated et metadonnees Airflow.
Redis	6379	Cache de l endpoint optimise.

2. Choix techniques

- Python est utilise pour l ingestion, les transformations et l API.
- MinIO et Elasticsearch couvrent la contrainte Raw avec stockage objet et recherche.
- PostgreSQL est choisi pour les couches structurees Staging et Curated.
- Airflow assure la reproductibilite et le scheduling du pipeline.
- FastAPI expose une interface claire pour consommer les donnees.
- Redis sert de cache optionnel pour les runs repetes de /ingest_fast.

3. Pipeline de donnees

Etape	Description	Sortie
ingest_file	Lit le dataset fichier financier.	Raw MinIO + Elasticsearch
ingest_api	Recupere les donnees recentes depuis Yahoo Finance.	Raw MinIO + Elasticsearch
transform_staging	Nettoie et calcule les indicateurs techniques.	staging_ohlcv
transform_curated	Detecte les anomalies via Isolation Forest.	curated_analysis
log_summary	Produit un resume de run dans Airflow.	Logs Airflow

4. Procedure d installation et de build

1. Installer et ouvrir Docker Desktop.
2. Se placer dans le dossier Projet-Data_Lake_finace.

3. Executer docker compose up -d --build.
4. Attendre que PostgreSQL, MinIO, Elasticsearch et Redis soient healthy.
5. Acceder a l API sur <http://localhost:8000/docs>.
6. Acceder a Airflow sur <http://localhost:8080> avec admin/admin.
7. Acceder a MinIO sur <http://localhost:9001> avec minioadmin/minioadmin.

5. Resultats de validation

Verification	Resultat
API /health	overall=ok
Airflow	scheduler healthy, metadatabase healthy
Import DAG	No data found
Tests unitaires	6 tests OK
DAG final	codex_finalcheck_20260707T1734Z success
Raw	533 objets MinIO et 5930 documents Elasticsearch
Staging	5899 lignes
Curated	4891 lignes

6. Performance du niveau avance

Batch	Standard	Optimise	Gain
1 ticker	4160.22 ms	1863.95 ms	55.20%
100 tickers	70631.54 ms	23612.12 ms	66.57%

Les gains sont superieurs au seuil attendu de 30%. Les optimisations principales sont la parallelisation, la vectorisation NumPy, l indexation bulk Elasticsearch, les batchs PostgreSQL et le cache Redis.

7. Points de vigilance

- Elasticsearch peut rester yellow en environnement local mono-noeud: ce n est pas bloquant.
- Airflow peut afficher un FutureWarning sur auth_backends avant Airflow 3.0: ce n est pas bloquant.
- Les performances yfinance varient selon le reseau et la disponibilite de Yahoo Finance.

8. Conclusion technique

La relance complete montre que le projet est fonctionnel. Les exigences de base et le niveau avance sont couverts par des preuves d execution, des tests unitaires et un benchmark reproductible.